



USC University of
Southern California

Midterm Review

CS 104

USC



What you need to know

- General Knowledge
- ADTs
- Runtime
- Inheritance/Polymorphism
- Linked List Recursion
- Data Structure Coding



Recognize these

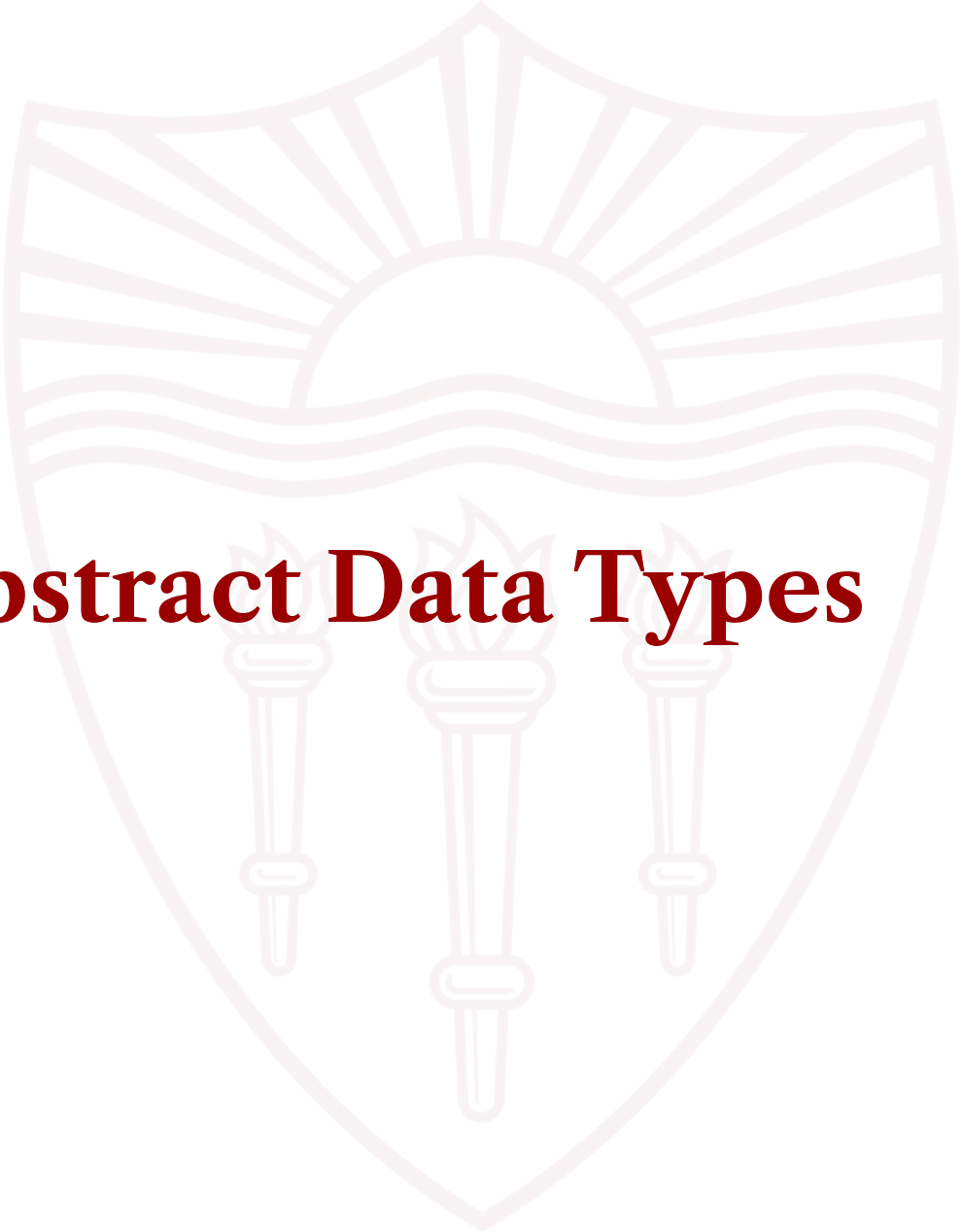
Arithmetic Series: $\sum_{i=1}^n \theta(i) = \theta(n^2)$

General Arith. Series: $\sum_{i=1}^n \theta(i^p) = \theta(n^{p+1})$

Harmonic Series: $\sum_{i=1}^n \theta(\frac{1}{i}) = \theta(\log(n))$

Geometric Series: $\sum_{i=0}^n \theta(c^i) = \frac{c^{n+1}-1}{c-1} = \theta(c^n)$



The background of the slide features a large, faint watermark of the University of South Carolina crest. The crest is a shield-shaped emblem. At the top, it depicts a rising sun with rays. Below the sun are three wavy lines representing water. The bottom half of the shield contains three torches, with the central torch being larger than the two flanking it.

REVIEW SECTION - Abstract Data Types

Overview



Type	Key Points	Operation Runtimes	Examples
List	Ordered, access based on position (index), may contain duplicates	Depends on implementation	A music album, a book series
Stack	LIFO (last in first out)	Push, pop, top All $O(1)$ if implemented right	A <i>stack</i> of trays at a cafeteria, tandem parking lot, “undo”
Queue	FIFO (first in first out)	Push back, pop front, front All $O(1)$ if implemented right	Waiting in line for a cashier at a store, printing a document
Map	Stores key, value pairs, keys must be unique (but values can have duplicates), no inherent ordering	Insert, remove, find/lookup: All $O(\log n)$	A dictionary, a grade book (key: student, value: grade)
Set	Like a map but only store keys, items must be unique, accessed based on keys, not indices	Insert, remove, find/lookup: All $O(\log n)$	Spices in a cupboard, all USC courses
Priority Queue	Self-orders items based on value, only allows retrieval/removal of the "best" priority item	Push, pop, top $O(\log n)$, $O(\log n)$, $O(1)$	A hospital where patients arrive with injuries of varying severity



ADTs

Scenario:

Track student ID $\rightarrow$ grade

What should we use?



ADTs

Scenario:

Track student ID $\rightarrow$ grade

What should we use?

NOT:

- list
- vector
- set



ADTs

Scenario:

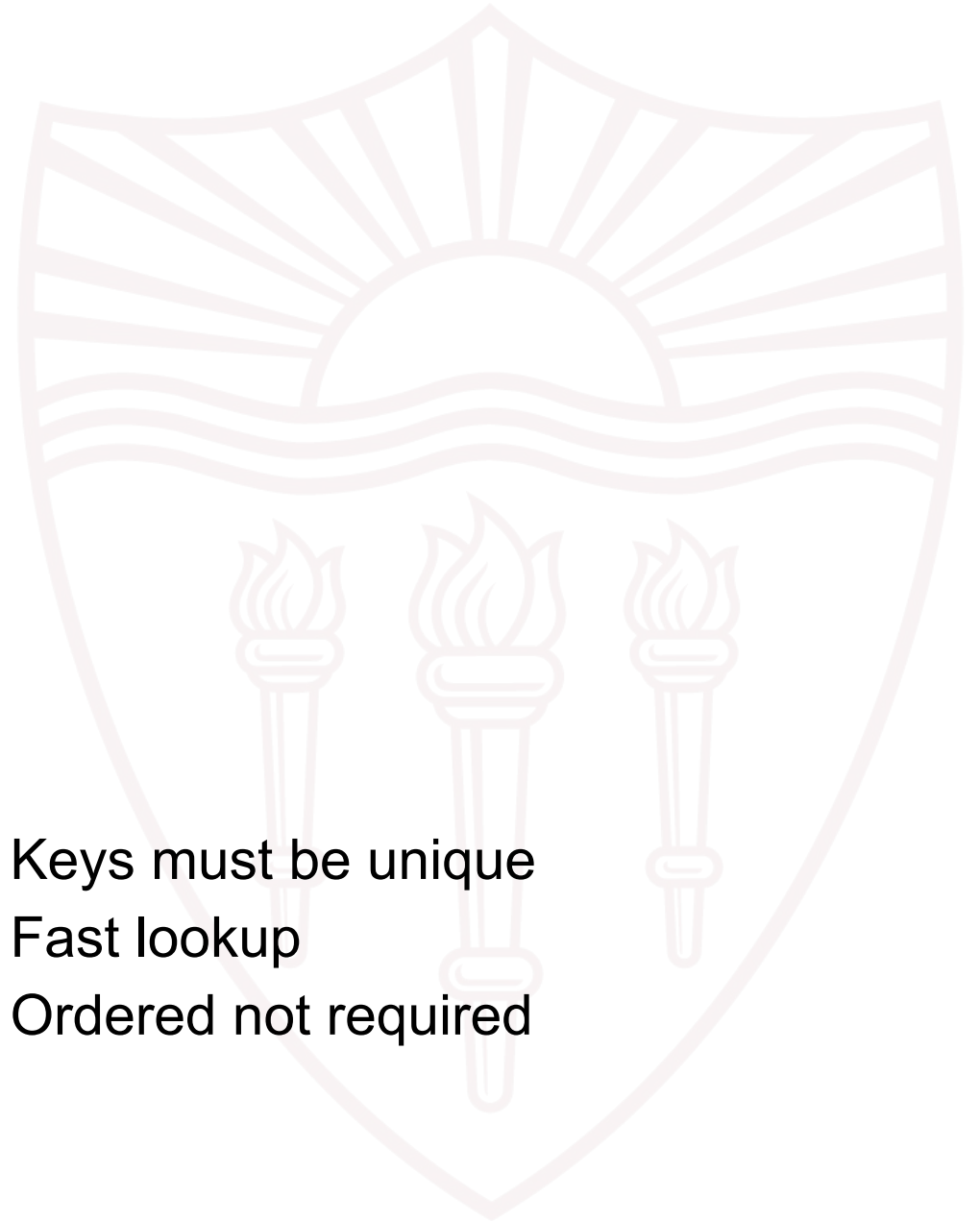
Track student ID $\rightarrow$ grade

What should we use?

NOT:

- list
- vector
- set

- Keys must be unique
- Fast lookup
- Ordered not required



ADTs

Scenario:

Track student ID → grade

What should we use?

NOT:

- list
- vector
- set



```
map<string, int>
```

- Keys must be unique
- Fast lookup
- Ordered not required

ADTs

Scenario:

Each student has a list of
homework scores

What should we use?



ADTs

Scenario:

Each student has a list of homework scores

What should we use?



```
map<string, vector<int>>
```

ADTs - Know these Runtimes

Vector:

- $\text{push_back} \rightarrow O(1)$
- $\text{insert/erase} \rightarrow O(n)$

Singly:

- $\text{push_front} \rightarrow O(1)$
- $\text{pop_back} \rightarrow O(n)$

Doubly:

- $\text{pop_back} \rightarrow O(1)$

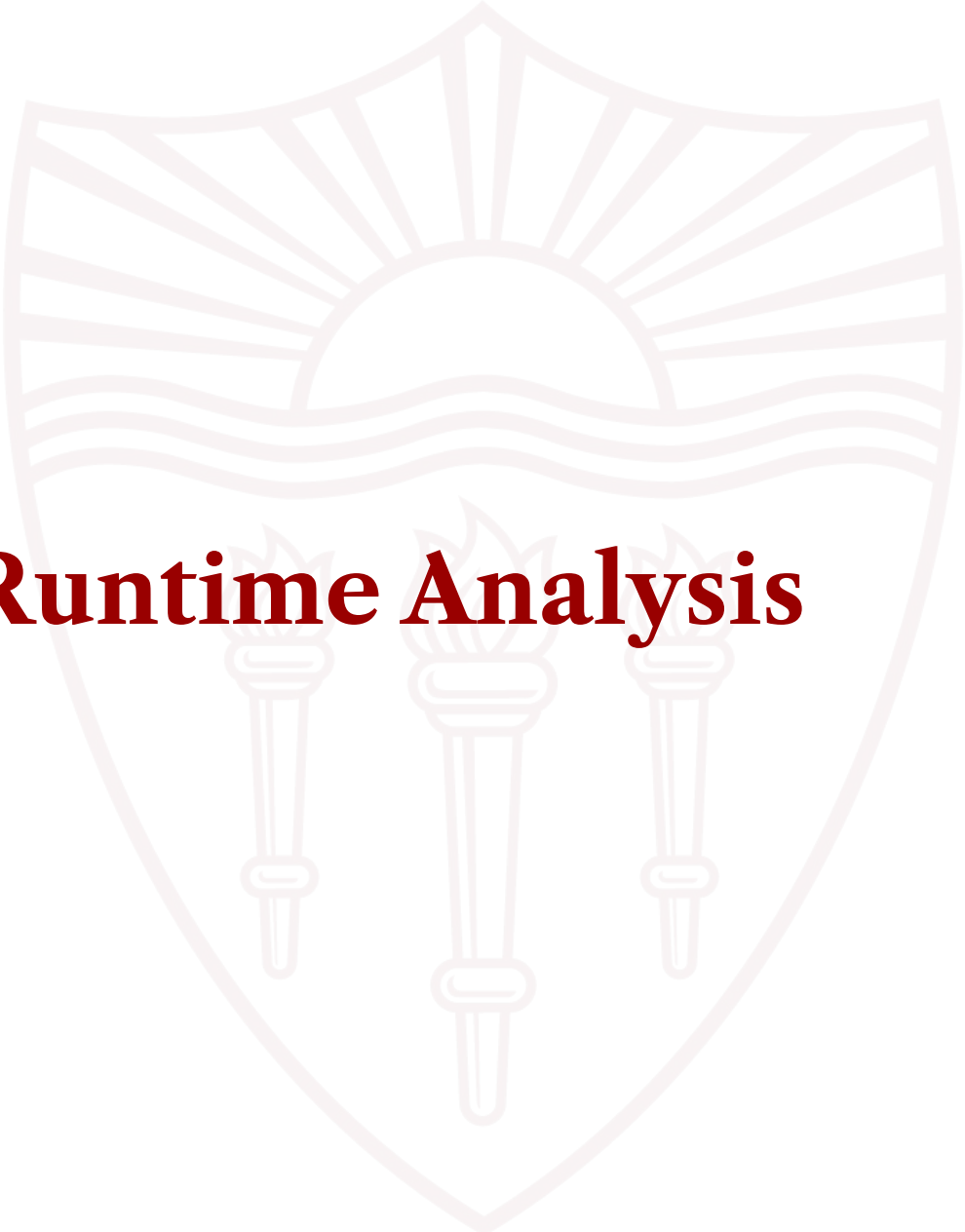
Set/Map:

- $\text{insert/find} \rightarrow O(\log n)$

Deque:

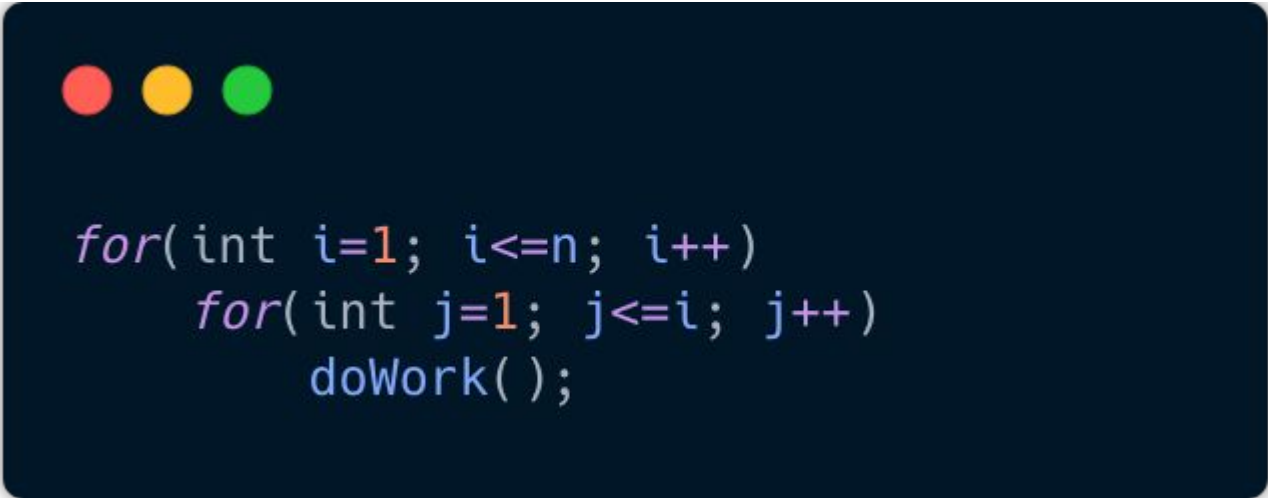
- everything $O(1)$





REVIEW SECTION - Runtime Analysis

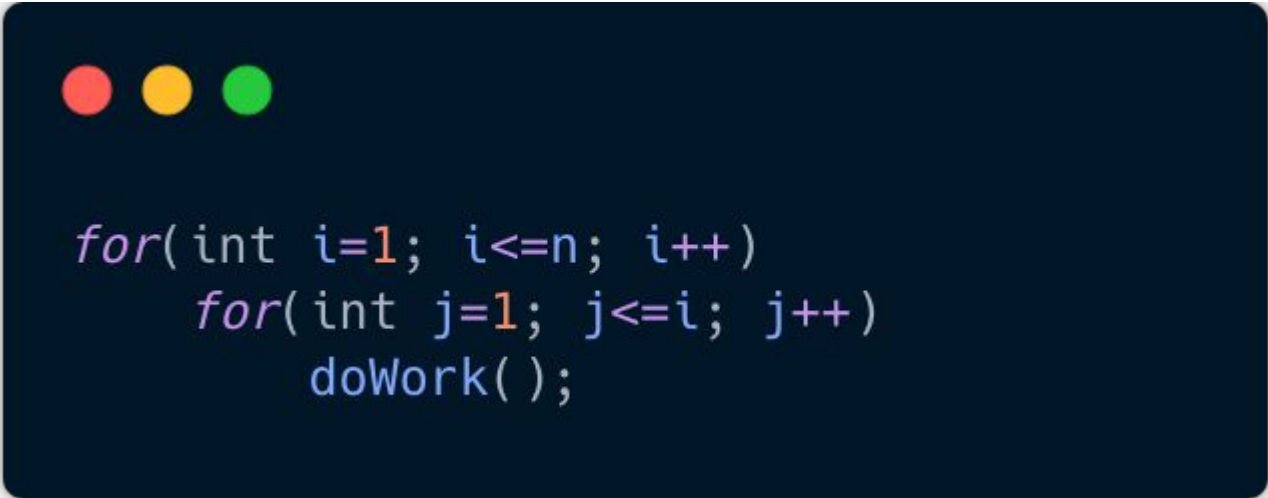
Runtime Analysis - Example



```
for(int i=1; i<=n; i++)  
    for(int j=1; j<=i; j++)  
        doWork();
```

PLZ SHOW WORK ON
EXAM

Runtime Analysis - Example

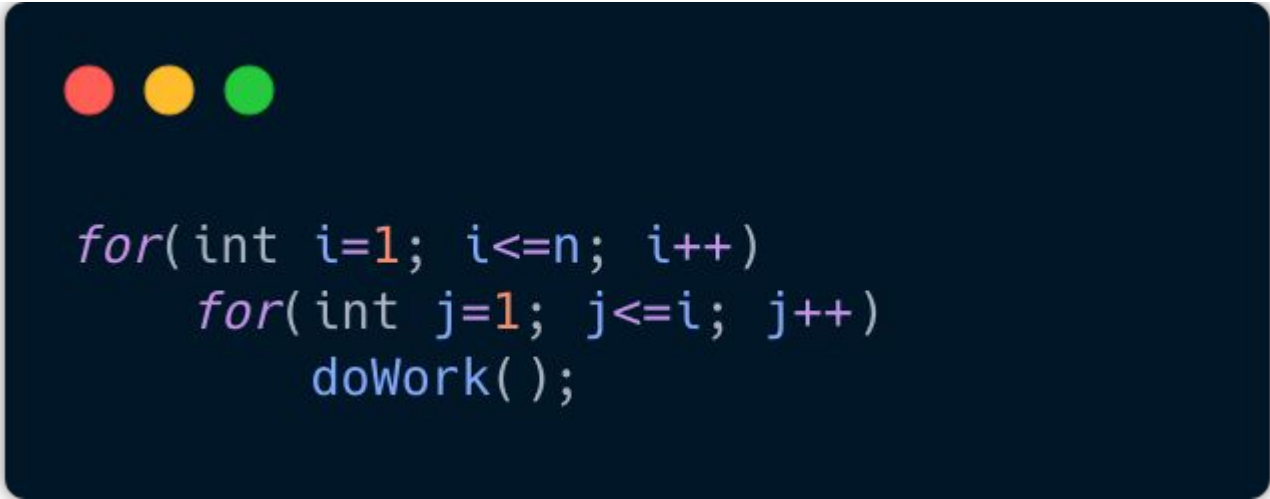


```
for(int i=1; i<=n; i++)  
    for(int j=1; j<=i; j++)  
        doWork();
```

Inner loop = i

Total = $\sum i$

Runtime Analysis - Example



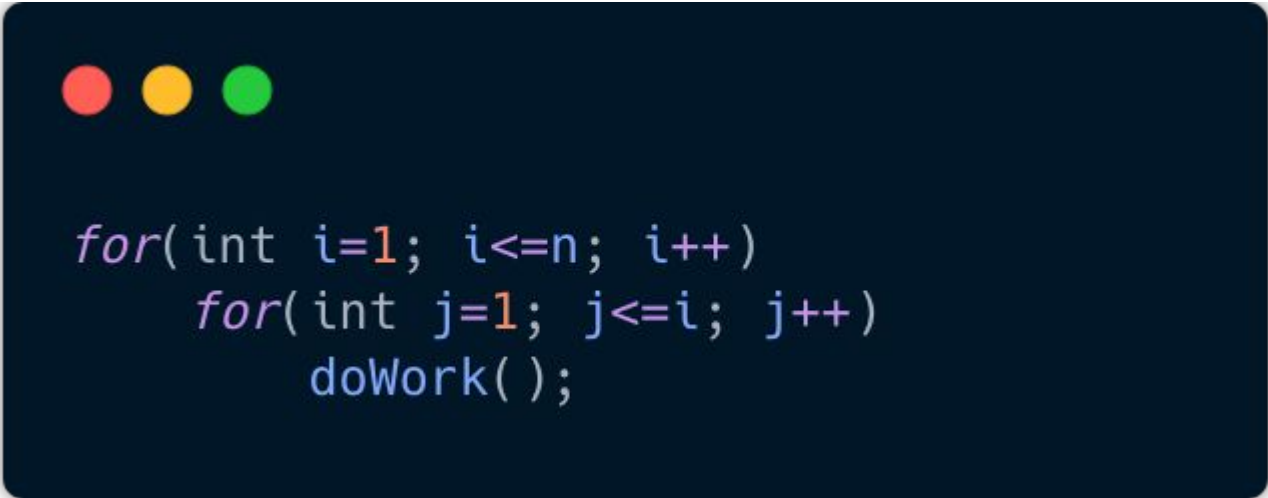
```
for(int i=1; i<=n; i++)  
    for(int j=1; j<=i; j++)  
        doWork();
```

Inner loop = i

Total = $\sum i$

= $n(n+1)/2$

Runtime Analysis - Example



```
for(int i=1; i<=n; i++)  
    for(int j=1; j<=i; j++)  
        doWork();
```

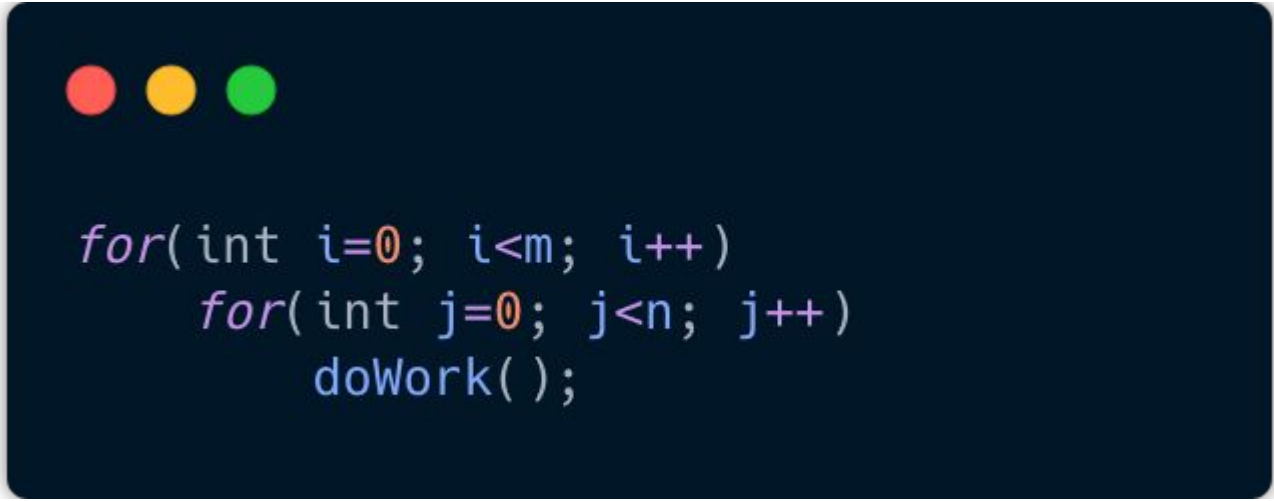
Inner loop = i

Total = $\sum i$

= $n(n+1)/2$

= $\Theta(n^2)$

Runtime Analysis - Example 2

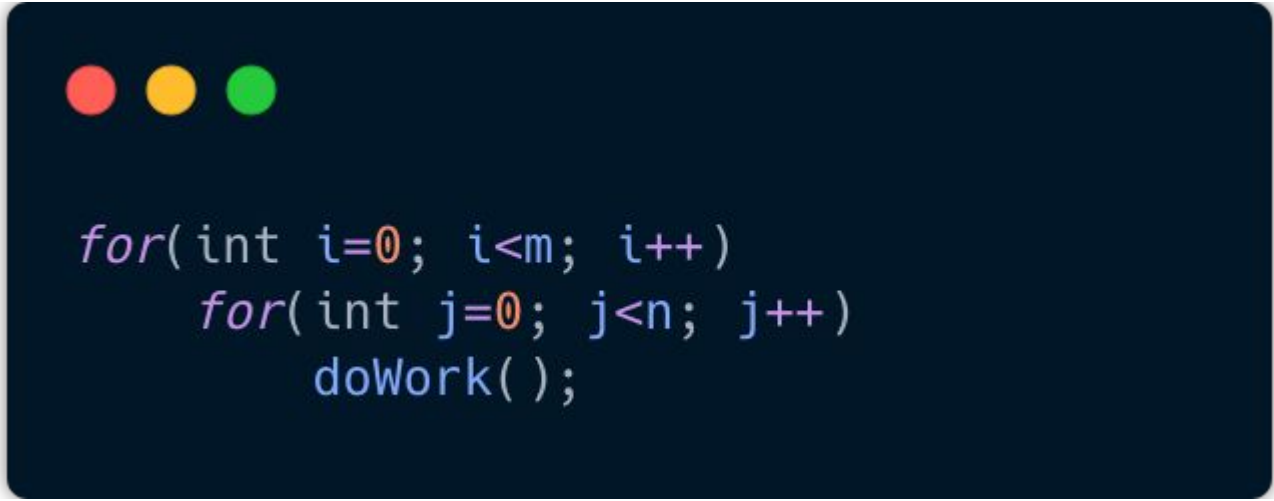


```
for(int i=0; i<m; i++)  
    for(int j=0; j<n; j++)  
        doWork();
```

2 different vars!

AHHHHHHH

Runtime Analysis - Example 2



```
for(int i=0; i<m; i++)  
    for(int j=0; j<n; j++)  
        doWork();
```

Step 1: Inner Loop

Inner loop runs:

n times

So for each fixed i, cost = $\Theta(n)$

Runtime Analysis - Example 2

```
for(int i=0; i<m; i++)  
    for(int j=0; j<n; j++)  
        doWork();
```

Step 2: Outer Loop

Outer loop runs m times.

Total work:

$$\sum_{i=1}^m n$$

Runtime Analysis - Example 2

```
for(int i=0; i<m; i++)  
    for(int j=0; j<n; j++)  
        doWork();
```

Step 3: Evaluate Summation

$= n + n + n + \dots + n$ (m times)

$= m * n$

$\Theta(mn)$

The background of the slide features a large, faint watermark of the University of South Carolina crest. The crest is a shield-shaped emblem. At the top, it depicts a rising sun with rays. Below the sun are three wavy lines representing water. At the bottom of the shield are three torches, one in the center and two on either side.

REVIEW SECTION - Inheritance and Polymorphism

Virtual vs. Non-Virtual

Which print() Gets Called?

1. "A"
2. "B"

```
class A {  
public:  
    virtual void print() { cout << "A"; }  
};  
  
class B : public A {  
public:  
    void print() { cout << "B"; }  
};  
  
int main() {  
    A* ptr = new B();  
    ptr->print();  
}
```


Virtual vs. Non-Virtual

Which print() Gets Called?

1. "A"

2. "B"

If function is virtual:

→ Use the **object type**

→ Calls B::print()

→ Output: **B**

```
class A {  
public:  
    virtual void print() { cout << "A"; }  
};  
  
class B : public A {  
public:  
    void print() { cout << "B"; }  
};  
  
int main() {  
    A* ptr = new B();  
    ptr->print();  
}
```

Public vs Protected vs Private



```
class Student {  
    private:  
        double mGpa;  
    protected:  
        string mMajor;  
    public:  
        string getName();  
};
```

- private: only this class
- protected: class + children
- public: interface

Public vs Protected vs Private - Example

What can be accessed?

```
class GeographicFeature {  
public:  
    virtual int GetArea() = 0;  
    std::string GetFeatureName();  
  
protected:  
    void setArea(int h);  
    void setFeatureName(std::string fn);  
  
private:  
    int area;  
    std::string featureName;  
};
```



```
class Island : public GeographicFeature {  
public:  
    std::string GetOcean();  
  
private:  
    std::string surroundingOcean;  
};  
  
class Atoll : public Island {  
public:  
    std::vector<std::string> GetCoralTypes();  
  
private:  
    std::vector<std::string> coralTypes;  
};
```

What can Atoll Access?

What can be accessed?

```
class GeographicFeature {  
public:  
    virtual int GetArea() = 0;  
    std::string GetFeatureName();  
  
protected:  
    void setArea(int h);  
    void setFeatureName(std::string fn);  
  
private:  
    int area;  
    std::string featureName;  
};
```



```
class Island : public GeographicFeature {  
public:  
    std::string GetOcean();  
  
private:  
    std::string surroundingOcean;  
};  
  
class Atoll : public Island {  
public:  
    std::vector<std::string> GetCoralTypes();  
  
private:  
    std::vector<std::string> coralTypes;  
};
```

What can Atoll Access?

What can be accessed?

```
class GeographicFeature {  
public:  
    ✓ virtual int GetArea() = 0;  
    ✓ std::string GetFeatureName();  
  
protected:  
    ✓ void setArea(int h);  
    ✓ void setFeatureName(std::string fn);  
  
private:  
    ✗ int area;  
    ✗ std::string featureName;  
};
```



```
class Island : public GeographicFeature {  
public:  
    ✓ std::string GetOcean();  
  
private:  
    ✗ std::string surroundingOcean;  
};  
  
class Atoll : public Island {  
public:  
    std::vector<std::string> GetCoralTypes();  
  
private:  
    std::vector<std::string> coralTypes;  
};
```

The background of the slide features a large, faint, light-colored crest of the University of South Carolina. The crest is a shield shape containing a rising sun with rays, wavy lines representing water, and three torches at the bottom.

REVIEW SECTION - Linked List, Recursion

Linked List Recursion

MENTAL CHECKLIST:

- ✓ No loops
- ✓ $O(n)$
- ✓ Visit each node once
- ✓ No memory leaks
- ✓ Return correct head

Base Case:



```
if(head == nullptr) return ...
```

Recursive call on smaller list



```
helper(head->next);
```

Do work either before OR after recursion

Work before → tail-style pattern

Work after → head-style pattern

Linked List Recursion

Base Case:



```
if(head == nullptr) return ...
```

Recursive call on smaller list



```
helper(head->next);
```

Do work either before OR after recursion

Work before → tail-style pattern

Work after → head-style pattern

ASK YOURSELF:

What information do I need to pass back up the recursion?

- A sum?
- A new head?
- A tail pointer?
- A boolean?

Linked List Recursion - Example

Write a recursive function that removes all nodes with value x from a linked list and return the new head.

$O(n)$. No loops plzzzzzz.

Example:

$[5, 2, 5, 3]$, $x = 5$

$\rightarrow [2, 3]$



Linked List Recursion - Step 1 - Base Case

```
Item* removeValue(Item* head, int x) {  
    // Base case: empty list  
    if(head == nullptr) {  
        return nullptr;  
    }  
  
    // recursive step will go here  
}
```

If the list is empty:

- Nothing to remove
- Return empty list

Linked List Recursion - Step 2 - Recurse

```
Item* removeValue(Item* head, int x) {  
    if(head == nullptr) {  
        return nullptr;  
    }  
  
    // Recursively fix the rest of the list first  
    head->next = removeValue(head->next, x);  
  
    // Now decide what to do with THIS node  
}
```

“Give me the
cleaned-up version of
everything after me.”

LL Recursion - Step 3 - Decide whether to keep this node



```
if(head->val == x) {  
    Item* temp = head->next;    // save the next node  
    delete head;              // prevent memory leak  
    return temp;               // skip this node  
}
```

If this node should be removed:

- Save its next pointer
- Delete current node (important!)
- Return the next node
- That effectively “skips” it

LL Recursion - Step 4 - Final Code

```
Item* removeValue(Item* head, int x) {  
  
    // 1 Base case: empty list  
    if(head == nullptr) {  
        return nullptr;  
    }  
  
    // 2 Recursively clean the rest of the list  
    head->next = removeValue(head->next, x);  
  
    // 3 If this node should be removed  
    if(head->val == x) {  
        Item* temp = head->next;    // save next node  
        delete head;               // free memory  
        return temp;               // skip current node  
    }  
  
    // 4 Otherwise keep this node  
    return head;  
}
```

MENTAL CHECKLIST:

- ✓ Each node visited once
- ✓ No loops
- ✓ $O(n)$
- ✓ Links updated correctly
- ✓ No memory leaks
- ✓ Correct head returned

Final Reminders

Think in Patterns

- Runtime → write the summation first.
- Linked list recursion → recurse on head->next, then fix yourself.
- Inheritance → apply access rules in two steps.
- ADTs → choose the most specific structure.
- Coding → think about memory.

If you're stuck

- Skip
- Come back
- Don't let one problem eat 20 minutes